

Data Science & AI Interview Questions — Hard Set

38 Hard Questions with Answers | ML Systems, Deep Learning, Algorithms, LLMs

1 Job Recommendation

HARD

Q Design a job recommendation system for a platform like LinkedIn. How do you handle cold start, freshness, and relevance simultaneously?

A Two-tower architecture: encode job (title, skills, company, seniority) and user (profile, experience, past applications, clicks) into a shared embedding space. Candidate retrieval via ANN (FAISS) on job embeddings filtered by recency (<30 days). Ranking: gradient-boosted model on features — embedding similarity, skill match score, location distance, salary range overlap, company size preference, application history. Cold start: new users → use explicit profile fields; new jobs → use content embeddings. Freshness: decay factor on job age; boost new postings. Multi-objective: balance relevance with application probability (not just clicks). Evaluate: application rate, interview rate (delayed signal). A/B test ranking changes. Re-rank for diversity (different industries, seniority levels).

2 Variable Error

HARD

Q You discover that a key feature in your deployed model has been computed incorrectly for the past 6 months due to a pipeline bug. How do you handle this?

A Immediate triage: (1) Quantify impact — compare correct vs incorrect feature values; measure how much model predictions changed on a sample. (2) Assess business harm — did incorrect predictions cause financial loss, bad decisions? (3) Fix the pipeline and backfill correct values in the feature store. Retraining: retrain the model on corrected historical data. Run shadow mode — serve both old and new model, compare outputs before switching. If retraining takes time, consider an interim patch: if the feature error is systematic (e.g., always off by a constant), apply a correction factor to raw model scores. Monitoring: add data validation checks (Great Expectations) and alerting on feature distribution drift to catch future bugs early. Post-mortem: document root cause, add unit tests for feature computation, implement feature-level monitoring.

3 KNN From Scratch

HARD

Q Implement K-Nearest Neighbors from scratch with support for weighted voting, and analyze the time/space complexity.

A Implementation: store all training data (X_{train} , y_{train}). For query point x : compute distances to all training points $d_i = \|x - x_i\|_2$. Find indices of k smallest distances. Weighted vote: $\text{weight}_i = 1/d_i$ (inverse distance); predicted class = argmax over classes of sum of weights for that class. Time complexity: $O(n \cdot d)$ per query (n training points, d features) — no training phase. Space: $O(n \cdot d)$ to store all points. Improvements: KD-tree reduces query to $O(d \cdot \log(n))$ for low dimensions but degrades to $O(n \cdot d)$ in high dimensions (curse of dimensionality). Ball tree works better for moderate dimensions. Approximate nearest neighbor (FAISS, HNSW) for large-scale. k selection: cross-validate; small $k \rightarrow$ overfitting, large $k \rightarrow$ underfitting. Feature scaling mandatory — use StandardScaler before KNN.

4

Car Recommendation Architecture

HARD

Q Design the ML architecture for a car recommendation system on an automotive marketplace.

A Multi-stage pipeline: (1) Hard filtering: user's stated constraints (price range, body type, fuel type, location radius) — structured DB query. (2) Collaborative filtering: users who viewed/saved similar cars also liked — ALS matrix factorization or two-tower neural model on interaction history. (3) Content-based: encode car features (make, model, year, mileage, features, price) and user preference profile as embeddings; retrieve by cosine similarity. (4) Contextual reranking: boost recently listed cars, penalize high-mileage for users who favor low-mileage, factor in financing pre-approval amount. (5) Diversity: ensure different makes/models in top-10. Cold start: use stated preferences from search history and filters. Evaluate: click-through rate, lead submission rate (contact dealer), purchase conversion. Delayed signal (purchase) requires attribution modeling.

5

Commercial Break

HARD

Q How would you build a model to determine the optimal timing and duration of commercial breaks in a streaming video platform?

A Frame as a reinforcement learning or optimization problem. Objective: maximize ad revenue while minimizing viewer churn caused by breaks. Features: video content (scene boundaries via shot detection, audio energy, subtitle density — avoid breaking mid-sentence), viewer engagement score (watch history, skip patterns), content genre, break position in video (early breaks hurt retention more), user subscription tier. Model: (1) Scene-boundary detector (computer vision) to find natural break points. (2) Churn risk model: predict $P(\text{user drops off} \mid \text{break inserted at time } t)$ using survival analysis or logistic regression on historical break experiments. (3) Optimization: given candidate break points and churn risk, maximize expected revenue = $\text{ad_value} * (1 - P(\text{churn}))$ subject to max 1 break per N minutes. A/B test break strategies; monitor completion rate and ad fill rate.

6

Type-ahead Search

HARD

Q Design a type-ahead (autocomplete) search system that returns relevant suggestions in under 50ms for millions of users.

A Architecture: (1) Offline: build a Trie or sorted prefix index over all query strings, weighted by historical query frequency * recency. Pre-compute top-k suggestions per prefix up to length 10 and cache in Redis. (2) Online: on each keystroke, look up current prefix in Redis → return pre-computed top-k in <5ms. (3) Personalization layer: re-rank suggestions using user's search history and location (e.g., 'star' → 'Starbucks near me' for a coffee drinker). (4) Freshness: streaming pipeline (Kafka) updates query counts in near-real-time; rebuild index nightly. (5) Spell correction: edit distance (BK-tree) for typo tolerance. (6) Scaling: shard Redis by prefix first letter; CDN-edge caching for popular prefixes. Latency budget: Redis lookup <5ms, personalization rerank <10ms, network <35ms = total <50ms.

7

Text-to-Image Retrieval

HARD

Q Design an end-to-end system at Amazon that retrieves relevant product images from a text description like 'red hiking backpack with water bottle holder'.

A Approach: contrastive vision-language model (CLIP-style). Offline: encode all product images with a vision encoder → store embeddings in a vector index (FAISS/ScaNN), sharded by product category. Online: encode user's text query with the text encoder (same embedding space) → ANN search retrieves top-k image embeddings → return corresponding products. Training: fine-tune CLIP on product-description pairs from Amazon catalog using contrastive loss (InfoNCE) — pulls matching image-text pairs together, pushes non-matching apart. Re-ranking: cross-encoder that jointly scores (text, image) pairs for precision. Cold start for new products: image embedding computed at listing time. Latency: ANN search <20ms. Evaluate: Recall@k on a held-out set of query-product pairs with human relevance labels.

8

Multi-Modal AI Tool

HARD

Q BCG is advising an e-commerce client on a multi-modal generative AI tool that processes product photos and text reviews to generate marketing copy. How do you handle multi-modal bias?

A Multi-modal vs unimodal: multi-modal model (e.g., GPT-4V, LLaVA) jointly processes image + text, capturing richer product context — better marketing copy but higher bias risk. Bias sources: (1) Training data bias — if product images predominantly show certain demographics, generated copy may reflect those assumptions. (2) Representation bias — certain product categories may be underrepresented. (3) Language bias — reviews from non-native speakers may be weighted differently. Mitigation: (1) Audit training data distribution across product categories, demographics, geographies. (2) Fairness metrics: measure output quality disparity across product segments. (3) Debiasing techniques: re-weighting training samples, adversarial debiasing. (4) Human review pipeline: sample generated copy for bias audit. (5) Diverse annotation team for evaluation. (6) Regular bias audits as data distribution shifts.

9

LLM Enterprise Search

HARD

Q Design an LLM-powered enterprise search system for a company with 10,000 employees and millions of internal documents (emails, PDFs, wikis).

A Architecture: (1) Ingestion: connectors for each source (Gmail, Confluence, SharePoint, Slack) → extract text, preserve metadata (author, date, access permissions). (2) Chunking: split documents into overlapping chunks (~500 tokens). Embed with a dense encoder (e.g., E5, BGE). Store in vector DB (Weaviate/Pinecone) alongside BM25 index for hybrid search. (3) Query processing: user query → hybrid retrieval (BM25 + dense) → rerank with cross-encoder → inject top-k chunks into LLM prompt → generate grounded answer with citations. (4) Access control: critical — only return chunks the querying user has permission to read (enforce ACL at retrieval time). (5) Freshness: incremental indexing as documents update. (6) Evaluation: answer accuracy on a golden QA set, retrieval MRR, latency. (7) Guardrails: no hallucination beyond retrieved context; cite sources. Monitor query logs for data leakage.

10

RAG Strict Source Control**HARD**

Q Design a RAG system where the LLM must only answer from ingested documents and never use its pretrained knowledge. How do you architect this?

A Strict grounding architecture: (1) Retrieval gate: always retrieve relevant chunks before generation — no fallback to parametric memory. (2) Prompt engineering: system prompt explicitly instructs: 'Answer ONLY using the provided context. If the answer is not in the context, say I don't know.' (3) Context sufficiency check: before generation, use a classifier or LLM call to verify retrieved context contains relevant information; if not, return 'insufficient information' rather than generating. (4) Post-generation verification: use an NLI model to check each sentence in the response is entailed by the retrieved context — flag or remove unsupported claims. (5) Fine-tuning: fine-tune the LLM on (context, question, grounded answer) triples to reinforce source-constrained behavior. (6) Confidence scoring: output confidence based on retrieval similarity scores. Evaluate with faithfulness metrics (RAGAS).

11

YouTube Recommendations**HARD**

Q Describe the architecture of a YouTube-scale video recommendation system.

A Two-stage deep learning system (as described in the YouTube DNN paper): (1) Candidate generation: a neural network takes user watch history, search history, demographics → produces ~hundreds of video candidates from millions. Uses approximate nearest neighbor on video embeddings. (2) Ranking: a deeper network scores the candidates on multiple objectives — predicted watch time (regression), click probability, like probability, share probability. Features: video embeddings, user context, freshness, channel authority, video quality signals. Multi-objective optimization: weighted combination to balance engagement vs satisfaction vs creator diversity. (3) Post-ranking: apply policy constraints (safe search, diversification, creator equity, avoid filter bubbles), freshness boost. (4) Exploration: epsilon-greedy or Thompson Sampling to explore new content and gather data. Evaluate: A/B tests on watch time per session, satisfaction surveys, 7-day retention.

12

Question Detection Ambiguity**HARD**

Q Design a system to distinguish questions from statements in a voice assistant for a language where both have identical word order and syntax.

A Multi-modal approach since syntax alone is insufficient: (1) Prosodic features: in most languages, questions have rising intonation (F0 pitch contour). Extract pitch trajectory, speaking rate, final syllable duration from audio using signal processing (librosa). Train a classifier on prosodic features. (2) Pragmatic/contextual features: conversation history (a question often follows a statement), turn-taking patterns, presence of question words even if syntactically optional. (3) Acoustic model: fine-tune a speech model (wav2vec 2.0) on labeled question/statement audio pairs to learn acoustic cues end-to-end. (4) Ensemble: combine prosodic classifier + acoustic model + dialogue context model. (5) Training data: collect labeled audio in the target language — crowdsource or use existing corpora. Evaluate with F1 on held-out audio. Handle regional accent variation via data augmentation.

13 Oversized Document Retrieval

HARD

Q At Intel, you need to retrieve and summarize information from technical documents that exceed the LLM's context window. Design the end-to-end system.

A Strategies for oversized documents: (1) Chunking + RAG: split document into overlapping chunks (e.g., 512 tokens, 50-token overlap). Embed all chunks. At query time, retrieve most relevant chunks via semantic search, inject into context window. Best for targeted Q&A.; (2) MapReduce summarization: split document into chunks → LLM summarizes each chunk independently (Map) → LLM combines summaries (Reduce). Good for global summarization. (3) Hierarchical chunking: chunk at sentence → paragraph → section level; retrieve at the right granularity for the query. (4) Long-context models: if budget allows, use models with 128k+ context (Claude, Gemini) for medium-length documents. (5) Selective extraction: use a cheap classifier to identify relevant sections before passing to LLM — reduces cost and latency. Evaluate: ROUGE and human evaluation on summary quality; answer accuracy on document QA benchmarks.

14 Logistic Regression from Scratch

HARD

Q Implement logistic regression from scratch using gradient descent and explain the math behind the update rule.

A Model: $P(y=1|x) = \text{sigmoid}(w^T x + b)$ where $\text{sigmoid}(z) = 1/(1+\exp(-z))$. Loss: binary cross-entropy $L = -[y \log(p) + (1-y) \log(1-p)]$. Gradient: $dL/dw = (1/n) * X^T * (p - y)$; $dL/db = \text{mean}(p - y)$. This elegant result comes from the chain rule: $dL/dp * dp/dz * dz/dw$, where the sigmoid derivative cancels with the log derivative. Update rule: $w = w - lr * dL/dw$; $b = b - lr * dL/db$. Implementation: initialize $w=0, b=0$. For each epoch: forward pass → compute $p = \text{sigmoid}(Xw+b)$; compute gradients; update weights. Add L2 regularization: $dL/dw += \lambda w$. Convergence: check loss decrease per epoch. Vectorized implementation is efficient with NumPy. Decision boundary: $w^T x + b = 0$ is a hyperplane. Multi-class: use softmax + cross-entropy (one-vs-rest or multinomial).

15 Design a Query-Retrieval System

HARD

Q Design a query-retrieval system for a Snowflake-like platform storing billions of SQL query records, supporting keyword search, metadata filtering, and relevance ranking.

A Architecture: (1) Indexing: ingest query records into Elasticsearch (or OpenSearch) — index SQL text with BM25 for keyword search; store structured metadata (user_id, warehouse, status, timestamps) as filterable fields. (2) Search: keyword query against SQL text field with BM25 scoring. Apply metadata filters as Elasticsearch filter clauses (fast, uses inverted index). (3) Ranking: combine BM25 relevance score (exact keyword boost) with recency score (decay function on execution_time). Expose top-k with cursor-based pagination. (4) Scale: shard Elasticsearch index by time range (monthly shards); warm shards for recent data, cold storage for old. Use index aliases for seamless rollover. (5) Performance: use Elasticsearch's filter cache for repeated metadata filters; pre-warm common queries. (6) Latency target: <200ms at p99 for 1B records. Capacity: ~100 bytes/record → ~100GB index with inverted index overhead.

16

RAG Hallucinations**HARD**

Q Identify failure points in a production RAG pipeline that cause hallucinations and design a multi-layered mitigation strategy.

A Failure points by stage: Retrieval: wrong chunks retrieved (low similarity, chunking boundary issues, embedding model mismatch) → model generates from weak context → hallucinates. Augmentation: too much irrelevant context confuses the model; context window overflow truncates key information. Generation: model ignores retrieved context and uses parametric memory; over-confident generation on uncertain queries. Mitigation layers: (1) Retrieval quality: hybrid search (BM25+dense), reranking with cross-encoder, adaptive chunking. (2) Context relevance filter: score retrieved chunks for query relevance; drop low-scoring chunks. (3) Prompt engineering: explicit grounding instruction; ask model to cite chunk IDs. (4) Post-generation faithfulness check: NLI model verifies each claim is entailed by retrieved context; flag unsupported sentences. (5) Confidence calibration: return 'I don't know' when retrieval similarity is below threshold. (6) Evaluation: RAGAS metrics (faithfulness, answer relevancy, context precision/recall).

17

Minimize Wrong Orders**HARD**

Q You work at a food delivery company. Design an ML system to minimize wrong or missing items in orders.

A Problem decomposition: (1) Predict high-risk orders: features — restaurant error rate history, order complexity (item count, special instructions count), time of day (busy periods), new menu items, courier rush. Train a binary classifier (wrong_order=1). Flag high-risk orders for additional verification. (2) Root cause analysis: cluster error types (missing item, wrong item, wrong quantity) by restaurant and menu item to identify systemic issues. (3) Real-time intervention: for high-risk orders, trigger a photo verification step by the courier (courier photographs bag contents). Use CV model to verify item count matches order. (4) Restaurant feedback loop: restaurants with high error rates get automated alerts, targeted training prompts in the restaurant app. (5) Menu intelligence: flag items frequently ordered incorrectly (confusable names, similar appearance). Evaluate: error rate reduction, customer complaint rate, refund rate. A/B test interventions.

18

k-Means from Scratch**HARD**

Q Implement k-Means clustering from scratch and explain convergence guarantees and failure modes.

A Algorithm: (1) Initialize k centroids randomly (or use k-Means++ for better initialization). (2) Assignment: assign each point to nearest centroid: $c_i = \arg\min_k \|x_i - \mu_k\|^2$. (3) Update: recompute each centroid as mean of assigned points: $\mu_k = \text{mean of } \{x_i : c_i = k\}$. (4) Repeat until assignments don't change. Convergence: guaranteed to converge — each step (assignment + update) monotonically decreases the within-cluster sum of squares (WCSS) objective; finite number of possible assignments → must terminate. However: converges to local minimum, not global. Solution: run multiple random initializations, keep best WCSS. Failure modes: (1) Sensitivity to initialization — k-Means++ mitigates by spreading initial centroids. (2) Assumes spherical, equal-size clusters — fails on elongated or unequal clusters. (3) Sensitive to outliers — use k-Medoids instead. (4) Must specify k — use elbow method or silhouette score to choose. (5) Fails in high dimensions — curse of dimensionality.

19

Random Forest from Scratch

HARD

Q Implement Random Forest from scratch and explain how bagging and feature randomness reduce variance.
:

A Algorithm: train B decision trees, each on a bootstrap sample (sample n points with replacement from training data) and at each split consider only m randomly selected features ($m \approx \sqrt{p}$ for classification). Prediction: majority vote (classification) or mean (regression) across all trees. Variance reduction via bagging: each tree is trained on different data $\rightarrow$ trees are decorrelated. By bias-variance decomposition: $\text{Var}(\text{mean of } B \text{ trees}) = \sigma^2/B + ((B-1)/B) \cdot \rho \cdot \sigma^2$ where ρ is pairwise tree correlation. Bagging reduces the first term; feature randomness reduces ρ (decorrelates trees) $\rightarrow$ overall variance decreases. Bias is unchanged (each tree is deep = low bias). Out-of-bag evaluation: $\sim 37\%$ of samples are never selected per tree $\rightarrow$ use as validation set for free. Feature importance: mean decrease in impurity (Gini) across all trees and splits. Permutation importance is more reliable for correlated features.

20

Proof k-Means Converges

HARD

Q Prove that the k-Means algorithm is guaranteed to converge.
:

A Proof sketch: Define the objective function $J = \sum \text{over all points of squared distance to assigned centroid}$: $J = \sum_i \|x_i - \mu_{c_i}\|^2$. Show J decreases monotonically at each step: Assignment step: for each point, we assign it to the nearest centroid. This can only decrease or maintain J — if a closer centroid exists, we switch; otherwise assignment stays the same. So $J(\text{after assignment}) \leq J(\text{before assignment})$. Update step: we recompute each centroid as the mean of its assigned points. The mean minimizes the sum of squared distances within a cluster (by calculus: $d/d(\mu_k) \sum \|x_i - \mu_k\|^2 = 0 \rightarrow \mu_k = \text{mean}$). So $J(\text{after update}) \leq J(\text{after assignment})$. Therefore J is non-increasing at every step. Since $J \geq 0$ (bounded below) and there are finitely many possible partitions of n points into k clusters, the algorithm must terminate in finite iterations. Note: it converges to a local minimum, not necessarily global.

21

Algorithm Reliability

HARD

Q How would you measure and improve the reliability of an ML algorithm in a production system?
:

A Reliability dimensions: (1) Accuracy stability: monitor prediction distribution, feature distributions (PSI), and business metrics over time. Alert on drift. (2) Infrastructure reliability: SLAs for latency (p50/p95/p99), error rate, uptime. Use circuit breakers — fall back to a simpler model or heuristic if the ML service is down. (3) Reproducibility: fix random seeds, version training data and model artifacts (MLflow/DVC). (4) Robustness to inputs: validate inputs at inference time (schema checks, range checks); handle missing features gracefully with defaults. (5) Shadow testing: run new model versions in shadow mode before promotion — compare outputs without affecting users. (6) Canary deployments: roll out to 1-5% of traffic first, monitor metrics, then expand. (7) Rollback plan: always keep the previous model version deployed and ready. (8) Load testing: verify model service handles peak traffic. (9) Explainability: SHAP values for debugging unexpected predictions.

22 Newsfeed Model

HARD

Q Design the ranking model for a social media newsfeed (like Facebook/LinkedIn).
:

A Multi-stage ranking: (1) Candidate retrieval: fetch recent posts from connections, followed pages, groups (~thousands). Lightweight scoring by recency and connection strength. (2) Feature extraction: post features (content type, topic, engagement velocity — likes/comments in first hour), author features (relationship strength to user, post quality history), user features (session context, time of day, device, recent interaction types), user-post interaction features (topic affinity, format preference). (3) Ranking model: multi-task neural network predicting: $P(\text{like})$, $P(\text{comment})$, $P(\text{share})$, $P(\text{click})$, $P(\text{hide/report})$. Weighted combination into final score. (4) Calibration: calibrate probabilities against observed rates. (5) Re-ranking: diversity (not 5 posts from same author), freshness boost, content policy enforcement, civic integrity labels. Evaluate: engagement rate, session depth, long-term satisfaction surveys (likes alone create outrage loops — balance with meaningful interactions).

23 Generating Discover Weekly

HARD

Q How would you design Spotify's Discover Weekly playlist generation system?
:

A Collaborative filtering core: (1) Build a user-song interaction matrix from streaming history. Apply matrix factorization (ALS) to learn latent user and song embeddings. Songs a user hasn't heard but are nearest to their taste profile in embedding space are candidates. (2) Audio content features: extract audio embeddings from raw audio (CNNs on mel spectrograms) — captures genre, tempo, mood, instrumentation. New songs with no streaming history get embeddings from audio. (3) NLP on playlist metadata: songs appearing together in user-created playlists are semantically related — use word2vec-style embedding on playlists. (4) Generation: for each user, retrieve top-k candidate songs (not heard recently), diversify across genres and moods, ensure 30 songs fit into a coherent listening arc (energy curve). (5) Personalization: weight candidates by user's recent listening trends (short-term taste shift). Evaluate: stream rate, full-listen rate, weekly return rate. Refresh weekly with latest interaction data.

24 Regularization and Validation

HARD

Q Explain L1 vs L2 regularization mathematically, and how validation helps you choose the regularization strength.
:

A L2 (Ridge): adds $\lambda \sum w_i^2$ to loss. Gradient: $2\lambda w$ — penalizes large weights, shrinks all weights toward zero but rarely to exactly zero. Bayesian interpretation: Gaussian prior on weights. Good when all features are expected to contribute. L1 (Lasso): adds $\lambda \sum |w_i|$ to loss. Subgradient at w_i : $\lambda \cdot \text{sign}(w_i)$ — can set weights exactly to zero → automatic feature selection. Bayesian: Laplace prior. Good for sparse models with many irrelevant features. Elastic Net: combines both. Choosing λ via validation: (1) Create a validation set (or use k-fold CV). (2) Train model with different λ values on the training set. (3) Evaluate each on validation set. (4) Select λ with best validation metric (e.g., lowest MSE, highest F1). (5) Retrain on full train+val with chosen λ , evaluate once on test. Never select λ based on test set — that's data leakage.

25

Generative Fraud Detection

HARD

Q Design a generative AI system to detect and explain financial fraud in online transactions where 3% are flagged as suspicious.

A Hybrid architecture: (1) Real-time detector: classical ML (XGBoost/LightGBM) for high-throughput, low-latency scoring on transaction features (amount, merchant, location, velocity, device). Handles 3% class imbalance with cost-sensitive learning. Output: fraud probability score + top SHAP features. (2) Generative explanation layer: for transactions flagged by the detector, pass fraud probability + SHAP features to an LLM (fine-tuned or prompted) to generate a human-readable explanation: 'This transaction was flagged because the amount (\$5,000) is 8x higher than the user's average, and the merchant is in a new country.' (3) Generative data augmentation: use a conditional GAN or VAE to synthesize realistic fraud transaction patterns for rare fraud types → augment training data. (4) RAG for investigator support: fraud analysts query a RAG system over historical fraud case reports to find similar cases. Evaluate: detector by PR-AUC; explanations by analyst satisfaction and accuracy.

26

Gradient Descent Calculation

HARD

Q Walk through a complete gradient descent update for a simple neural network with one hidden layer on a single training example.

A Setup: input x , hidden layer $h = \text{ReLU}(W1 \cdot x + b1)$, output $y_{\text{hat}} = \text{sigmoid}(W2 \cdot h + b2)$, loss $L = \text{binary cross-entropy}$. Forward pass: $z1 = W1 \cdot x + b1$; $h = \text{ReLU}(z1)$; $z2 = W2 \cdot h + b2$; $y_{\text{hat}} = \text{sigmoid}(z2)$; $L = -[y \cdot \log(y_{\text{hat}}) + (1-y) \cdot \log(1-y_{\text{hat}})]$. Backward pass (chain rule): $dL/dy_{\text{hat}} = (y_{\text{hat}} - y) / (y_{\text{hat}} \cdot (1-y_{\text{hat}}))$; $dy_{\text{hat}}/dz2 = y_{\text{hat}} \cdot (1-y_{\text{hat}})$; so $dL/dz2 = y_{\text{hat}} - y$ (elegant cancellation). $dL/dW2 = dL/dz2 \cdot h^T$; $dL/db2 = dL/dz2$; $dL/dh = W2^T \cdot dL/dz2$. Through ReLU: $dL/dz1 = dL/dh \cdot (z1 > 0)$ [element-wise — zero where $z1 \leq 0$]. $dL/dW1 = dL/dz1 \cdot x^T$; $dL/db1 = dL/dz1$. Update: $W1 \leftarrow W1 - lr \cdot dL/dW1$; $W2 \leftarrow W2 - lr \cdot dL/dW2$. Repeat for all examples (SGD) or accumulate over mini-batch.

27

Related Jobs Optimization

HARD

Q How would you optimize a 'Related Jobs' feature on a job platform to maximize application rates?

A Problem: given a job a user is viewing, surface the k most relevant jobs they might also apply to. Approach: (1) Content similarity: encode jobs as embeddings (title, skills, seniority, industry) using a sentence transformer. Related jobs = nearest neighbors in embedding space, filtered by same seniority level and location. (2) Co-application signal: jobs that the same users frequently apply to together are implicitly related — train an item2vec or collaborative filtering model on application sequences. (3) Behavioral signals: click-through and application rates on related jobs shown → train a ranking model to maximize $P(\text{apply} \mid \text{related job shown})$. (4) Diversity: avoid surfacing jobs from the same company or identical titles — ensure variety. (5) Personalization: re-rank related jobs by user's profile match score. Evaluate: CTR on related jobs, application rate, downstream interview rate. A/B test against baseline (pure content similarity).

28

Quantify Uncertainty

HARD

Q How do you quantify and communicate prediction uncertainty in an ML model?
:

A Methods: (1) Bayesian approaches: Bayesian neural networks maintain a distribution over weights → sample multiple predictions, compute mean and variance. MC Dropout: apply dropout at inference time, run N forward passes, compute prediction mean and std. (2) Conformal prediction: model-agnostic framework that produces calibrated prediction intervals with guaranteed coverage (e.g., 90% CI contains true value 90% of the time). (3) Ensemble uncertainty: train N models (or use random forest) — variance across model predictions estimates uncertainty. (4) Calibration: for classifiers, check if $P(\text{correct} \mid \text{confidence}=p) \approx p$ using reliability diagrams. Calibrate with temperature scaling or isotonic regression. (5) Epistemic vs aleatoric uncertainty: epistemic (model uncertainty, reducible with more data) vs aleatoric (inherent noise, irreducible). Deep ensembles can decompose these. Communication: report prediction intervals alongside point estimates; flag high-uncertainty predictions for human review in safety-critical applications.

29

Dog Rescue Robot

HARD

Q You are designing the perception and decision-making ML system for a robot that rescues dogs in disaster zones. What models would you use?
:

A Perception stack: (1) Object detection: fine-tuned YOLO or DETR on dog + debris + survivor images. Use thermal/infrared camera for low-light conditions. (2) Semantic segmentation: identify traversable terrain, obstacles, collapsed structures. (3) Depth estimation: stereo camera or LiDAR for 3D spatial awareness. (4) Dog state classification: predict dog's injury severity from visual cues (posture, movement) to prioritize rescue order. (5) SLAM (Simultaneous Localization and Mapping): map the disaster zone while navigating. Decision-making: (1) Reinforcement learning agent (trained in simulation — IsaacGym) for navigation policy: given sensor observations, output movement commands. Sim-to-real transfer via domain randomization. (2) Rescue prioritization: score each detected dog by injury severity, accessibility, battery level remaining. (3) Human-in-the-loop: uncertain decisions escalate to remote human operator. Safety constraints: never enter structurally unstable zones without sensor confirmation.

30

Kernel Methods

HARD

Q Explain kernel methods in SVMs. What is the kernel trick and why is it powerful?
:

A SVMs find a maximum-margin hyperplane in feature space. For non-linearly separable data, map inputs to a high-dimensional feature space $\phi(x)$ where they become separable. Problem: $\phi(x)$ can be infinite-dimensional (e.g., RBF maps to infinite dimensions). The kernel trick: the SVM optimization and prediction only require inner products $\phi(x_i)^T \phi(x_j)$, never $\phi(x)$ itself. A kernel function $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ computes this inner product directly in the original space — no explicit mapping needed. Common kernels: Linear: $K = x^T x$. Polynomial: $K = (x^T x + c)^d$. RBF/Gaussian: $K = \exp(-\gamma \|x - x'\|^2)$ — infinite-dimensional feature space, local similarity. Sigmoid. Mercer's theorem: any positive semi-definite function is a valid kernel. Powerful because: enables non-linear classification at the computational cost of a linear SVM. Weakness: $O(n^2)$ kernel matrix computation — doesn't scale to millions of samples.

31 Trending Sort

HARD

Q Design a ranking algorithm for 'trending' content that balances recency, velocity, and quality.
:

A Classic approaches: (1) Hacker News formula: $\text{score} = (\text{upvotes} - 1) / (\text{age_hours} + 2)^{\text{gravity}}$, where $\text{gravity} \approx 1.8$.
: Penalizes age, rewards votes. Simple and interpretable. (2) Reddit's Wilson score for ranking by quality; decay separately for trending. (3) Custom: $\text{score} = f(\text{velocity}) * g(\text{age}) * h(\text{quality})$. Velocity: engagement rate in last 1h vs baseline (z-score). Age decay: exponential decay $e^{-(\lambda * \text{age})}$. Quality: upvote ratio, creator credibility score. Multi-objective extension: separate trending scores for different time windows (last 1h, 6h, 24h) to surface both flash trends and sustained trends. Personalization: re-weight trending score by user's topic affinity. Anti-gaming: detect coordinated upvote rings (velocity anomaly detection). Evaluate: CTR on trending content vs chronological; freshness of trending items; diversity of topics.

32 Hotel Occupancy Prediction

HARD

Q Build a model to predict hotel room occupancy for the next 30 days for revenue management.
:

A Time-series forecasting problem. Features: historical occupancy (lag 7, 14, 28 days), booking pace (reservations made for the target date as of today — most predictive signal), day of week, holidays, local events (concerts, conferences scraped from Eventbrite), seasonality, room type, competitor pricing (web scraped), cancellation rate trend, weather forecast. Model: (1) Baseline: same day last year * growth trend. (2) LightGBM with all features — fast, handles interactions, works well with tabular time-series. (3) Prophet for interpretable seasonality decomposition. (4) Evaluate with MAE and MAPE on rolling 30-day backtests; segment by room type and market segment. Output: occupancy probability distribution per day, not just point estimate. Use forecast for dynamic pricing (RevPAR optimization). Retrain weekly with updated booking data. Alert on forecast vs actual deviation >10% for human review.

33 Clustering Basketball Players

HARD

Q How would you cluster NBA players into archetypes using ML?
:

A Feature engineering: per-100-possession stats (points, assists, rebounds, steals, blocks, turnovers), shooting profile (3PA rate, mid-range rate, rim rate, 3P%, TS%), on/off court impact, defensive metrics (DBPM, opponent FG% at rim), usage rate, assist-to-usage ratio, height, weight, position. Preprocessing: standardize all features (StandardScaler); apply PCA to reduce to 10-15 principal components (capturing ~90% variance) while removing multicollinearity. Clustering: K-Means with $k=8-10$ (determined by elbow + silhouette); alternatively HDBSCAN for density-based clusters that don't force spherical shapes. Validation: check cluster coherence by eyeballing named examples — do clusters correspond to known archetypes (3-and-D wings, point centers, playmaking guards, stretch 4s, rim runners)? Silhouette score for quantitative validation. Visualize with UMAP 2D projection, colored by cluster. Use clusters for team-building analytics and trade analysis.

34 Infer Location from Activity

HARD

Q Given a user's anonymized activity logs (app usage, time patterns, movement speed), how would you infer their home and work locations?

A Approach: (1) Home location: users are most consistently at home late night (11pm-6am). Cluster GPS pings during these hours using DBSCAN (handles arbitrary shapes, noise). The densest cluster = home. (2) Work location: cluster activity between 9am-6pm on weekdays. Largest non-home cluster = likely workplace. (3) Features for clustering: dwell time (stationary >30 min), visit frequency, recurrence pattern (same location on multiple weekdays). (4) Movement inference: if no GPS, infer from app activity patterns — high messaging activity correlated with commute times, productivity apps during work hours. Use HMM (Hidden Markov Model) to model state transitions (home → commute → work). (5) Confidence scoring: require minimum N days of consistent signal before labeling. (6) Privacy: all processing on-device where possible; k-anonymity and differential privacy before any aggregation. Validate: compare inferred vs reported home zip code (if available) on a test set.

35 Clustering With Mixed Variable Types

HARD

Q How do you cluster a dataset with both continuous and categorical variables?

A Challenge: standard K-Means uses Euclidean distance, which is not meaningful for categorical variables. Solutions: (1) Gower distance: computes a composite distance that handles mixed types — for continuous: normalized absolute difference; for categorical: indicator of inequality (0/1). Use with hierarchical clustering or PAM (Partitioning Around Medoids). (2) K-Prototypes: extends K-Means for mixed data. Centroids have means for continuous and modes for categorical features. Distance = Euclidean for continuous + Hamming for categorical, weighted by gamma (tune gamma). (3) Embedding approach: encode categoricals with embeddings (learned or target encoding) → treat all features as continuous → apply K-Means. (4) Multiple Correspondence Analysis (MCA): like PCA for categorical data; apply to categoricals, combine with continuous PCA components, then cluster. (5) UMAP: can handle mixed distance matrices. Best practice: normalize continuous features, choose gamma in K-Prototypes via silhouette score.

36 Adam Optimizer

HARD

Q Explain the Adam optimizer mathematically and why it often outperforms vanilla SGD.

A Adam (Adaptive Moment Estimation) maintains per-parameter adaptive learning rates using two moment estimates: $m_t = \beta_1 * m_{t-1} + (1-\beta_1) * g_t$ (first moment / mean of gradients). $v_t = \beta_2 * v_{t-1} + (1-\beta_2) * g_t^2$ (second moment / uncentered variance). Bias correction: $\hat{m}_t = m_t / (1-\beta_1^t)$; $\hat{v}_t = v_t / (1-\beta_2^t)$ — corrects for zero initialization at start. Update: $\theta = \theta - lr * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$. Typical values: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$. Why better than SGD: (1) Adaptive learning rates per parameter — sparse features get larger updates (similar to AdaGrad). (2) Momentum (first moment) smooths gradient noise and accelerates convergence. (3) Works well with minimal tuning. Weaknesses: can converge to sharp minima (worse generalization than SGD with momentum on some tasks); high memory use (2x parameter storage). AdamW fixes weight decay implementation (decouples from gradient scaling).

37

Separate Models for Age Groups

HARD

Q When would you build separate models for different user age groups vs one model with age as a feature?
:

A One model with age as feature: simpler, more data per model, better for interpolation across ages, easier to maintain.
: Sufficient when age has a smooth, monotonic relationship with the target and interactions with other features are limited. Separate models per age group: preferred when: (1) The relationship between features and target fundamentally differs by age group — different features matter (a credit model for college students uses different signals than for retirees). (2) Non-linear age interactions are complex and a single model can't capture them well. (3) You have enough data per segment for reliable training. (4) You can afford the operational overhead of multiple models. Middle ground: one model with age interaction features (age * feature_X terms), or a hierarchical model. Evaluation: compare single vs segmented models on held-out data per age group using your primary metric. If segmented models outperform significantly on specific groups, split those; otherwise keep unified.

38

Optimizing Threshold Adjustment in Default Risk Models

HARD

Q Your credit default prediction model produces probabilities. How do you choose the optimal classification threshold for loan approval decisions?
:

A Business framing: threshold choice involves a direct cost-benefit tradeoff. Define: cost of false negative (FN) = approved loan that defaults → expected loss ($EL = PD * LGD * EAD$). Cost of false positive (FP) = rejected loan that would have repaid → lost revenue (net interest margin over loan term). Optimal threshold: classify as default if $P(\text{default}) > \tau$ where τ minimizes expected total cost: $\tau^* = \text{cost_FN} / (\text{cost_FN} + \text{cost_FP})$. This equals the ratio of the opportunity cost to the sum of both costs. Implementation: (1) Plot cost curve across all thresholds on a validation set with natural class distribution. (2) Choose τ^* at minimum expected cost. (3) Segment thresholds by loan type, amount, or customer segment — a \$50k loan may warrant different τ than a \$500 personal loan. (4) Fairness check: ensure threshold doesn't create disparate impact across protected groups (ECOA compliance). (5) Backtesting: simulate approval decisions on historical data, compute realized loss rates. Monitor threshold performance monthly as macro conditions change.